Tutorial 05

▼ Overview

Tutorial

- Balancing Trees
- AVL Trees
- Graph Representation
- A Tutors Worst Nightmare (Code Style)

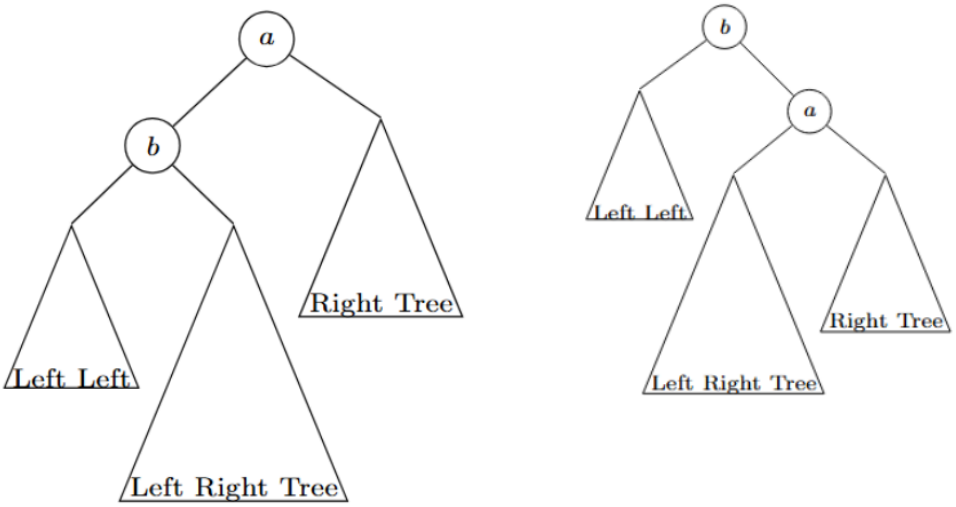
Lab (From Page)

- To explore an application of graphs (Continuing on the Tutorial)
- To get some practice with graph problems (Continuing on Tutorial)
- To perform complexity analysis on graph algorithms (Continuing on Tutorial + Practice Last Week and Tutorial 2)
- To implement some basic features of social networks (Continuing the Tutorial)

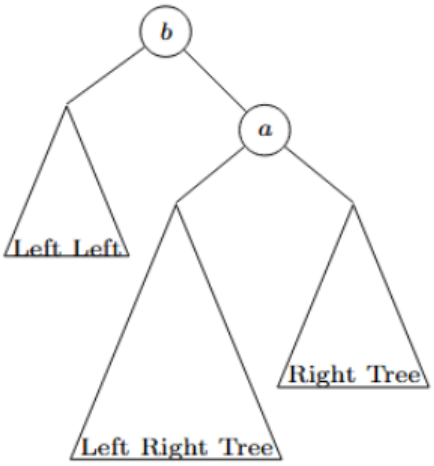
▼ Balancing BSTs

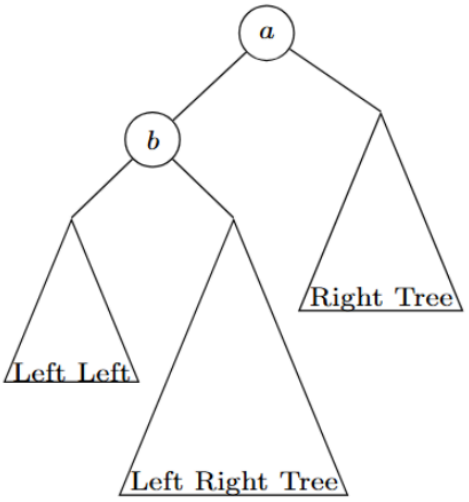
▼ Rotations Reminder

- Left and right rotations are as suggested moving the current node to be the left or right subtree and then replacing it with the node to the left but we need to maintain order.
- A Right Rotation:



- A Left Rotation:

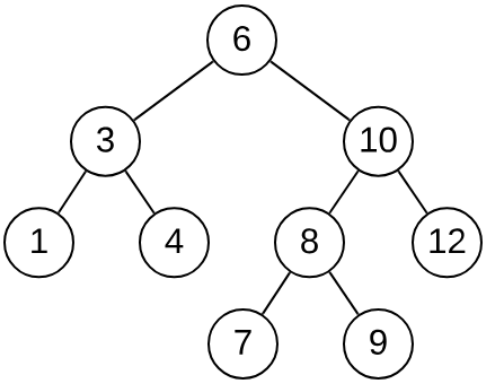


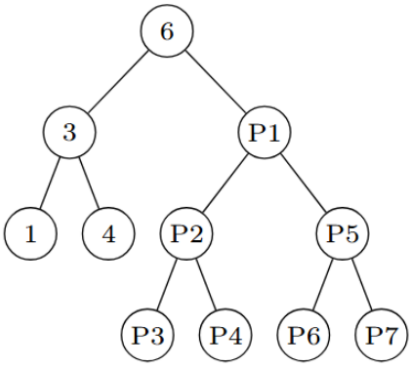


▼ Do a right rotation on the node containing 10 followed by a left rotation on the node containing 6.

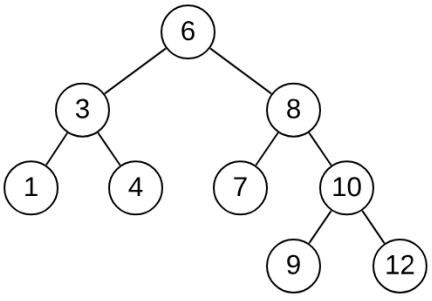
▼ Not OS

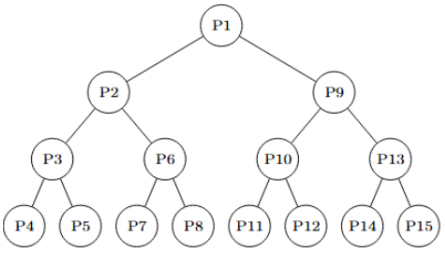
- Rotate 10 Right (2 Circles Each)





▼ Rotate 6 Left (2 Circles Each)



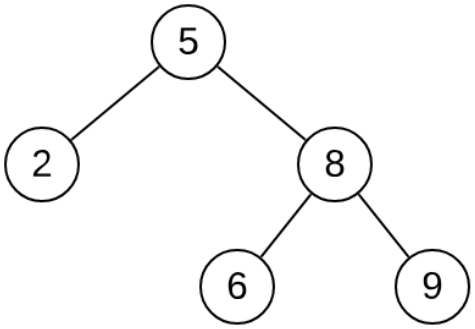


▼ Insert-At-Root Meaning

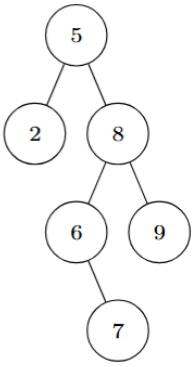
This is referring to the act of inserting the node at the root while rotating up to the root.

▼ Show the result of performing the insert-at-root operation with the value 7 in this binary search tree:

▼ Not OS



▼ Where is it inserted normally



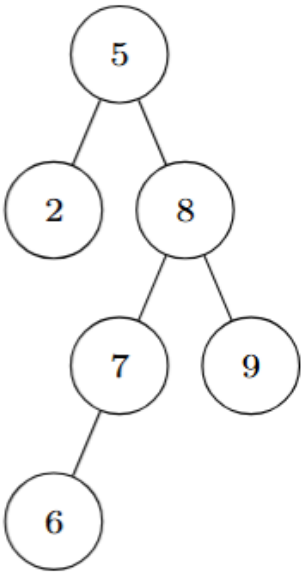
▼ How many rotations do we need?

3 Rotations

▼ What is the first rotation and where?

Rotate left on node 6

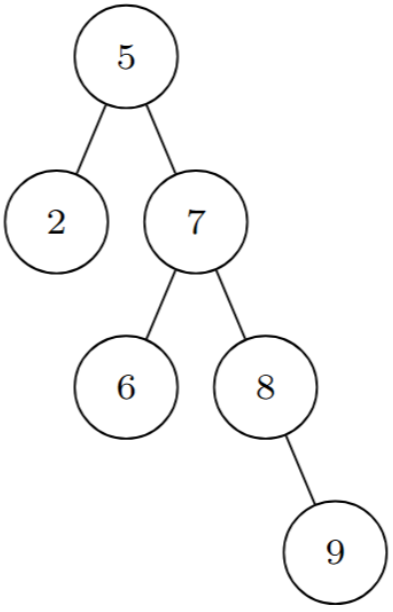
▼ What does that look like?



▼ What is the second rotation and where?

Rotate right on node 8

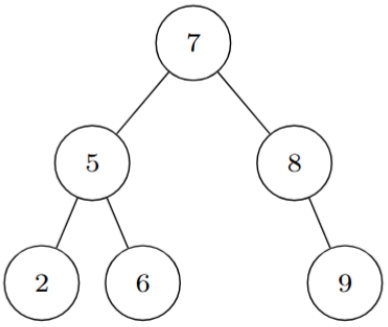
▼ What does that look like?



▼ What is the third rotation and where?

Rotate left on node 5

▼ What does that look like?



▼ AVL Trees

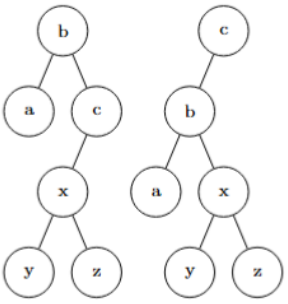
AVL trees use the following node definition:

```
struct node {
    int data;
    int height;
    struct node *left;
    struct node *right;
};
```

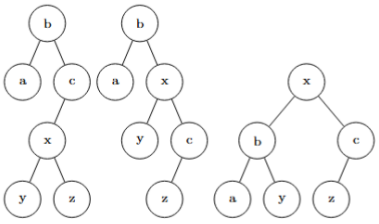
▼ For the following AVL tree, annotate each node with the height of its subtree.

▼ Inserting in AVL Trees

- When inserting into an AVL Tree we want to ensure that that height order of the child nodes is abs(height(node→left) -height(node→right)) < 1 e.g.
  - If there are no children -1 +1 = 0
  - As soon as the difference is greater than 1 between height. e.g. 1 to -1, calculations 1 + 1 or -1 -1 which is clearly greater than 1 either way.
- Special Case Right Rotation:
  - Right Side Has More Height on the Left
    - Trying Normal Rotation

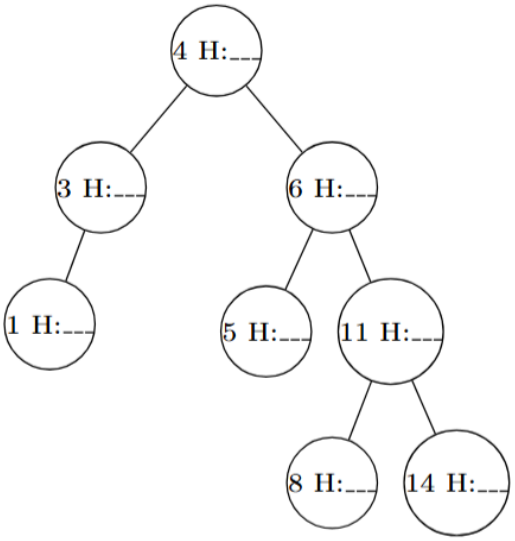


    - Need to Rebalance Rights node rotating left first

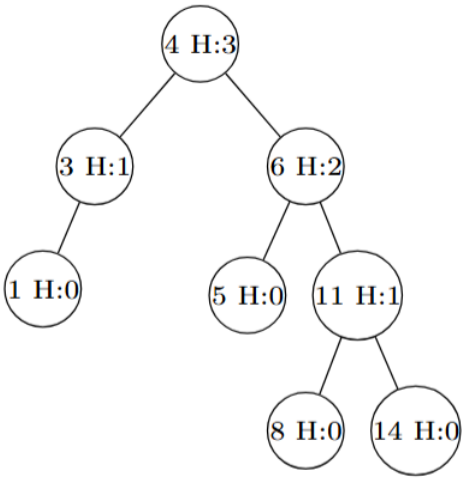


- Special Case Left Rotation:
  - For the same reason the left child must be rotated right if the height is not equal favoring the right of the left child node.

▼ Label the Heights

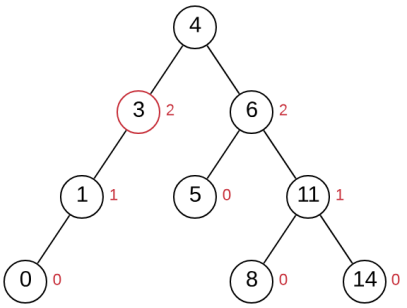


▼ Answer

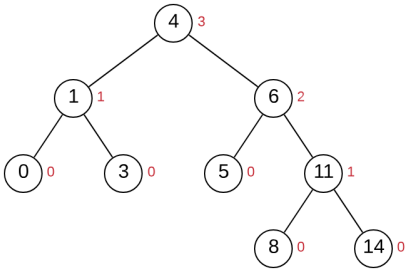


▼ Insert 0

▼ Normal Location Where?



▼ Rotation 1 Is this a Special Case?

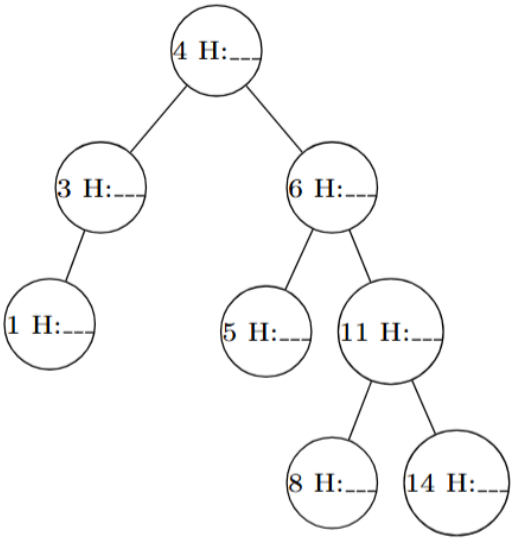


More Rebalance?

▼ Rotation 2 Is this a Special Case?

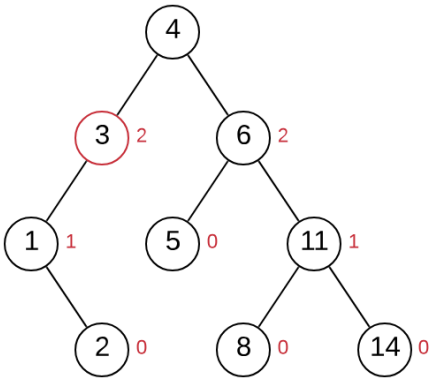
More Rebalance?

▼ Rotation 3 Is this a Special Case?

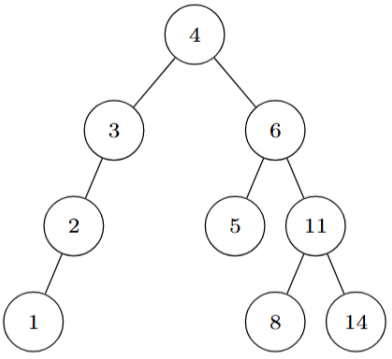


▼ Insert 2

▼ Normal Insert Where?

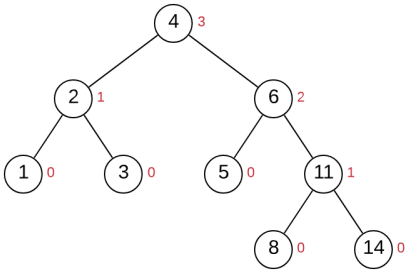


▼ Rotation 1 Is this a Special Case?



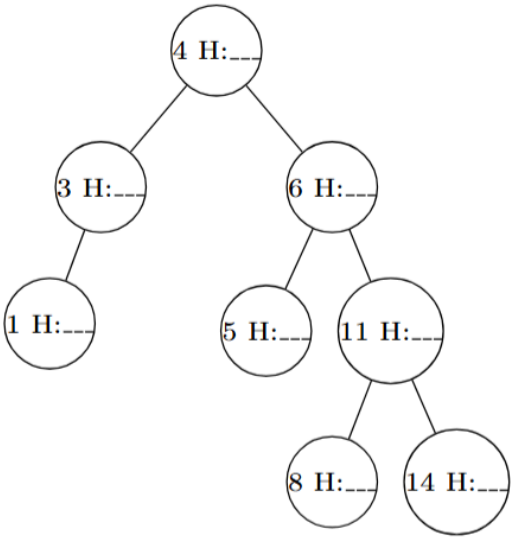
More Rebalance?

▼ Rotation 2 Is this a Special Case?

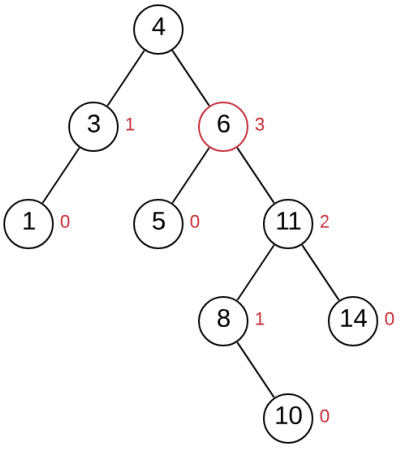


More Rebalance?

▼ Rotation 3 Is this a Special Case?



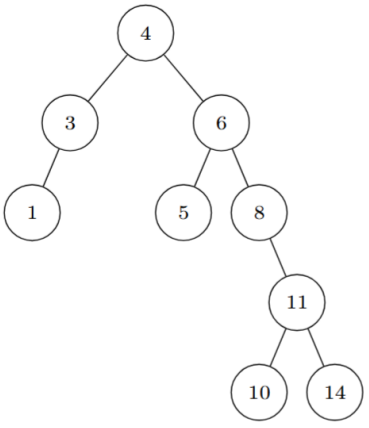
- Insert 10
  - Normal Insert Where?



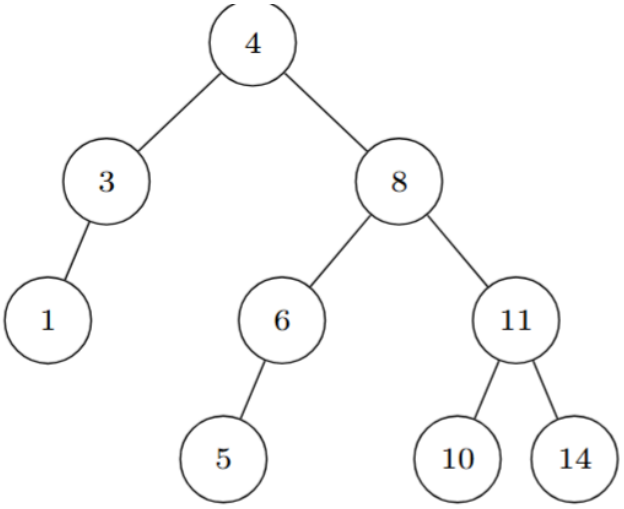
  - Rotation 1 Is this a Special Case?

    Yes

    - Rotation Step 1



▼ Rotation Step 2

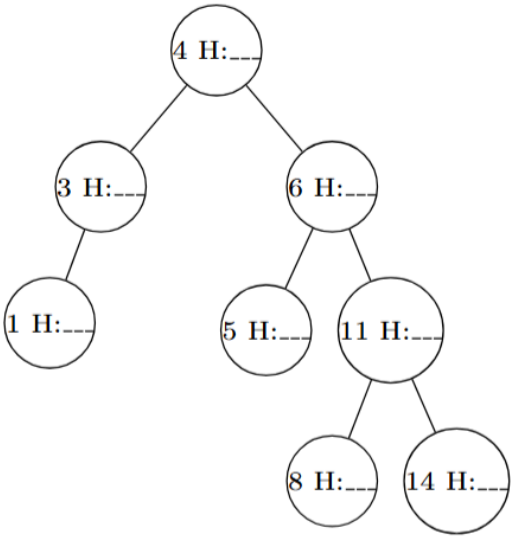


More Rebalance?

▼ Rotation 2 Is this a Special Case?

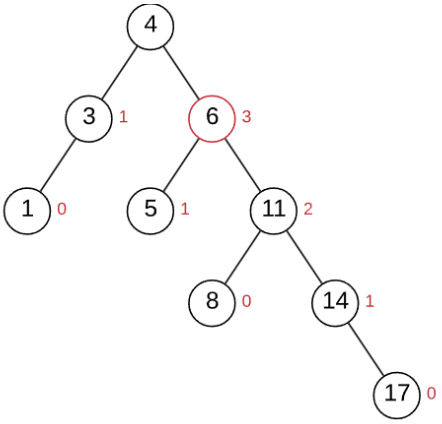
More Rebalance?

▼ Rotation 3 Is this a Special Case?

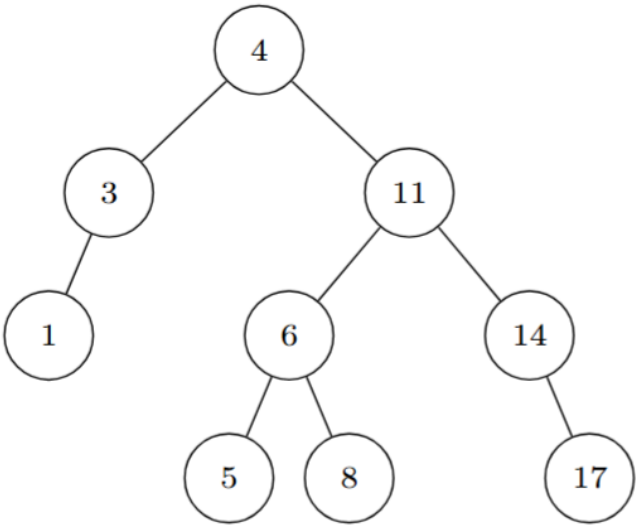


▼ Insert 17

▼ Normal Insert Where?



▼ Rotation 1 Is this a Special Case?



More Rebalance?

▼ Rotation 2 Is this a Special Case?

More Rebalance?

▼ Rotation 3 Is this a Special Case?

▼ Create This AVL Tree From Scratch?

▼ Not on the screen

▼ Insert 12 Inside

▼ Normal Location Where?

▼ Rotation 1 Is this a Special Case?

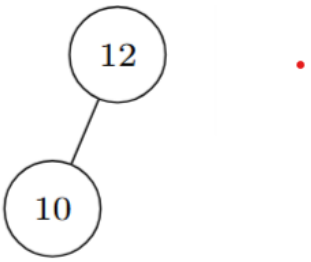
More Rebalance?

▼ Rotation 2 Is this a Special Case?

More Rebalance?

▼ Rotation 3 Is this a Special Case?

- Insert 10 inside
  - Normal Location Where?



  - Rotation 1 Is this a Special Case?

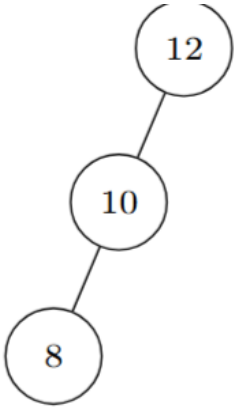
  More Rebalance?

  - Rotation 2 Is this a Special Case?

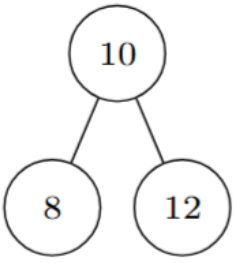
  More Rebalance?

  - Rotation 3 Is this a Special Case?

- Insert 8 inside
  - Normal Location Where?



  - Rotation 1 Is this a Special Case?



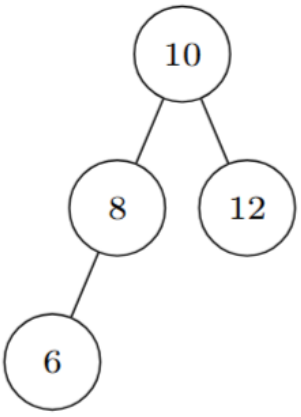
  More Rebalance?

  - Rotation 2 Is this a Special Case?

  More Rebalance?

  - Rotation 3 Is this a Special Case?

- Insert 6 inside
  - Normal Location Where?



  - Rotation 1 Is this a Special Case?

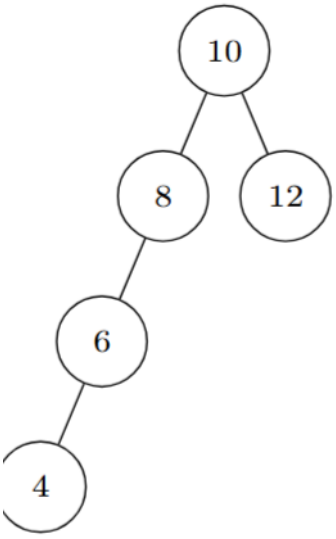
  More Rebalance?

  - Rotation 2 Is this a Special Case?

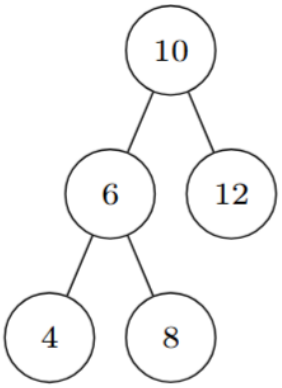
  More Rebalance?

  - Rotation 3 Is this a Special Case?

- Insert 4 inside
  - Normal Location Where?



  - Rotation 1 Is this a Special Case?



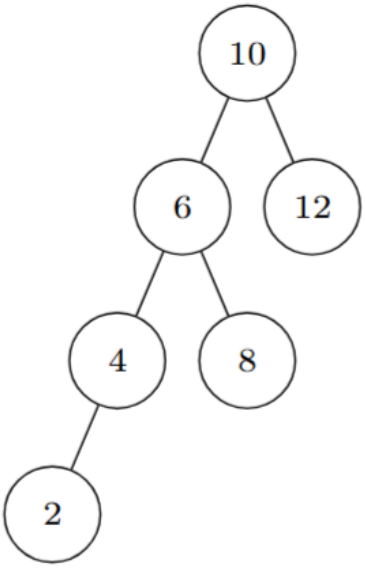
More Rebalance?

- ▼ Rotation 2 Is this a Special Case?

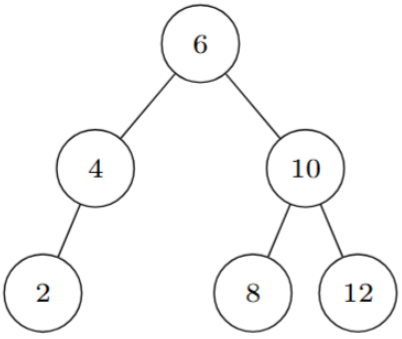
More Rebalance?

- ▼ Rotation 3 Is this a Special Case?

- ▼ Insert 2 inside
  - ▼ Normal Location Where?



  - ▼ Rotation 1 Is this a Special Case?



More Rebalance?

▼ Rotation 2 Is this a Special Case?

More Rebalance?

▼ Rotation 3 Is this a Special Case?

▼ Using an AVL tree, design an algorithm to determine if an array contains two elements that sum to a given value.

1. Write the algorithm in pseudocode.

```
hasTwoSum(A,v)
    Input:
    Output:

    # Checks Maybe???

    # Code Goes Here
```

▼ A Solution

```
hasTwoSum(A,v):
    Input:  array A[0..n - 1] of integers
            integer v
    Output: true if A contains two elements that sum to v, false otherwise

    if v is even and v/2 occurs more than once in A then
        return true
    end if

    t = new AVL tree
    for i = 0 up to n - 1 do
        if 2 x A[i] =/= v then
            insert A[i] into t
        end if
    end for

    for i = 0 up to n - 1 do
        if (v - A[i]) is in t then
            return true
        end if
    end for

    return false
```

▼ Cleaner Solution

```
hasTwoSum(A, v):
    Input:  array A[0..n - 1] of integers
            integer v
    Output: true if A contains two elements that sum to v, false otherwise
```

```
    t = new AVL tree

    for i = 0 up to n - 1 do
        if (v - A[i]) is in t then
            return true
        end if
        insert A[i] into t
    end for

    return false
```

2. Analyse the algorithm

▼ A Solution

- First if (loop) O(n) since counting if contains
- AVL tree insertion O(nlogn) because n potential inserts at O(logn) an insert.
- Now n searches at O(logn) so O(nlogn)

▼ Final Complexity

O(nlogn)

Faster than Week 3.

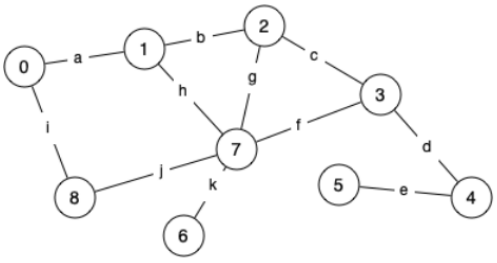
▼ Cleaner Solution

- First loop n searches and n insertions into an AVL tree

▼ Final Complexity

O(nlogn)

▼ Graph Representation



▼ How Many Edges?

11

▼ How Many Cliques

▼ What is a clique?

Complete Subgraph with At Least 5 Nodes

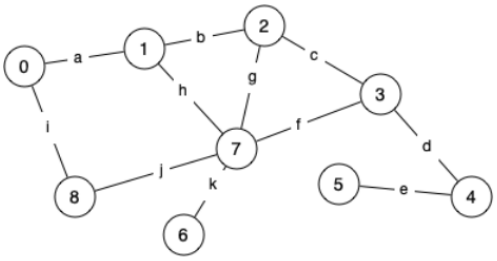
2

- {1, 2, 7},
- {2, 3, 7}

▼ How Many Cycles?

6 Cycles

- 0-1-7-8-0,
- 1-2-7-1,
- 2-3-7-2,
- 0-1-2-7-8-0,
- 0-1-2-3-7-8-0,
- 1-2-3-7-1



▼ What is the degree of each vertex?

- d(5,6) = 1,
- d(0,4,8) = 2,
- d(1,2,3) = 3
- d(7) = 5

▼ How many edges in the longest path from 5 to 8?

- Remember path has no repeating edges.

7 edges,

- 5-4-3-2-7-1-0-8
- 5-4-3-7-2-1-0-8

▼ What is an adjacency matrix?

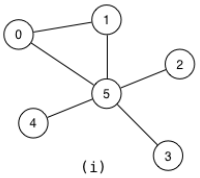
A matrix (table like structure) with a 1 where a connection occurs in a graph and 0 where it doesn't

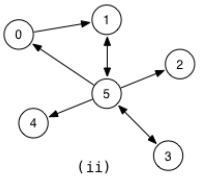
▼ What is an adjacency list representation?

A representation where the nodes are represented as a double linked list.

▼ What is an list of edges representation?

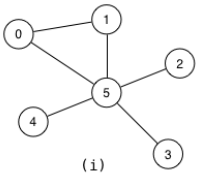
A list where each contains a tuple (two element structure) that contains the to node and from node

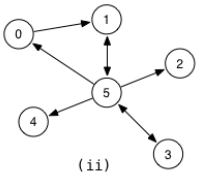




▼ Finish this Adjacency Matrix Representation (i)

	0	1	2	3	4	5
0						
1						
2						
3						
4						
5						

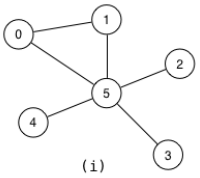


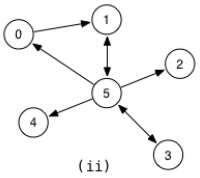


▼ Finish this Adjacency Matrix Representation (ii)

	0	1	2	3	4	5
0						

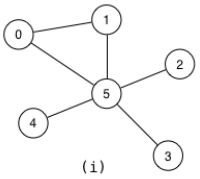
1						
2						
3						
4						
5						

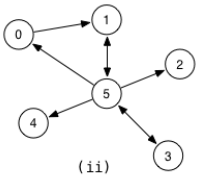




▼ Finish this Adjacency List Representation (i)

0						
1						
2						
3						
4						
5						





▼ Finish this Adjacency List Representation (ii)

0						
1						
2						
3						
4						
5						

▼ How is the adjacency matrix for a directed graph different to that for an undirected graph?

The adjacency matrix for an undirected graph is symmetric and has zeroes on the leading diagonal (no self-edges). The adjacency matrix for a directed graph is typically not symmetric and can have non-zero values on the leading diagonal (self-edges are allowed).

▼ Facebook could be considered as a giant "social graph”

▼ What are the vertices?

Vertices are people (Facebook users)

▼ What are edges?

Edges are "friend" relationships

▼ Are the edges directional?

No - if you are someone's friend, they are also your friend

▼ What does the degree of each vertex represent?

The degree of a vertex is the number of friends that the person represented by the vertex has

▼ What kind of graph algorithm could suggest potential friends?

Breadth-first search - find your immediate friends, then consider their friends, and so on

▼ A Tutor's Worst Nightmare

Fix the style

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void freeList(struct Node *head) {
    if (head == NULL) {
    return;
    }
    // Frees the list by iterating thro
ugh the list, using a temporary element
and freeing the elements by using free
(hopefully this comment is useful and h
elps out)
    struct Node *Temp = head;
    while (head != NULL) {
        Temp = head;
        head = head->next;
        free(Temp);
    }
}
void print_list(struct Node *node) {
    while (node->next != NULL) {
        printf("%d -> ", node->data);
        node = node->next;
    }
    printf("%d\n", node->data);
    return;
}
struct Node *add(struct Node *a, int da
ta) {
    // malloc the node
    struct Node *newNode = (struct Node
*)malloc(sizeof(struct Node));
    // set the node's data field
    newNode->data = data;
    // set the node's next field to NUL
L
    newNode->next = NULL;
    if (a == NULL) {
        a = newNode;
    } else {
        struct Node *lastNode = a;
```

▼ Answers

This is a list of most of the style issues in the code:

- Inconsistent indentation
  - The `return` statement in `freeList` is not indented
  - Both tabs and spaces used for indentation
- Lack of vertical whitespace
  - There is no whitespace between functions
  - There is no whitespace between different logical parts of a function, e.g., the `add` function
- Incorrect function order
  - Helper functions should be placed under the functions that use them, in the order that they are used.
  - In the program, we can see that the `main` function uses `add()`, then `print_list()`, then `freeList()`, so the `main` function should appear first, followed by `add()`, `print_list()` and then `freeList()`.
- Poor commenting
  - Each function should have a comment above it describing its purpose
  - Comments should be used to provide explanation and commentary, not to restate the code
- Inconsistent naming style
  - A consistent naming style should be used for variable and function names. The allowed styles are `camelCase` and `snake_case`, and only one should be used across the entire program.
- Poor variable/function names
  - `add` is too short of a variable name and doesn't adequately describe the function. A better name would be `appendList`.

```
        while (lastNode->next != NULL)
{
            lastNode = lastNode->next;
        }
        lastNode->next = newNode;
    }
    return a;
}
int main() {
    struct Node *head = NULL;
    head = add(head, 1);
    head = add(head, 2);
    head = add(head, 3);
    head = add(head, 4);
    print_list(head);
    freeList(head);
    return 0;
}
```

- `a` is not a meaningful variable name. A better name might be `list` or `listHead`.

- Unnecessary type casts
  - Type casts are not necessary when using `malloc`.
- Code order
  - Completely wrong.